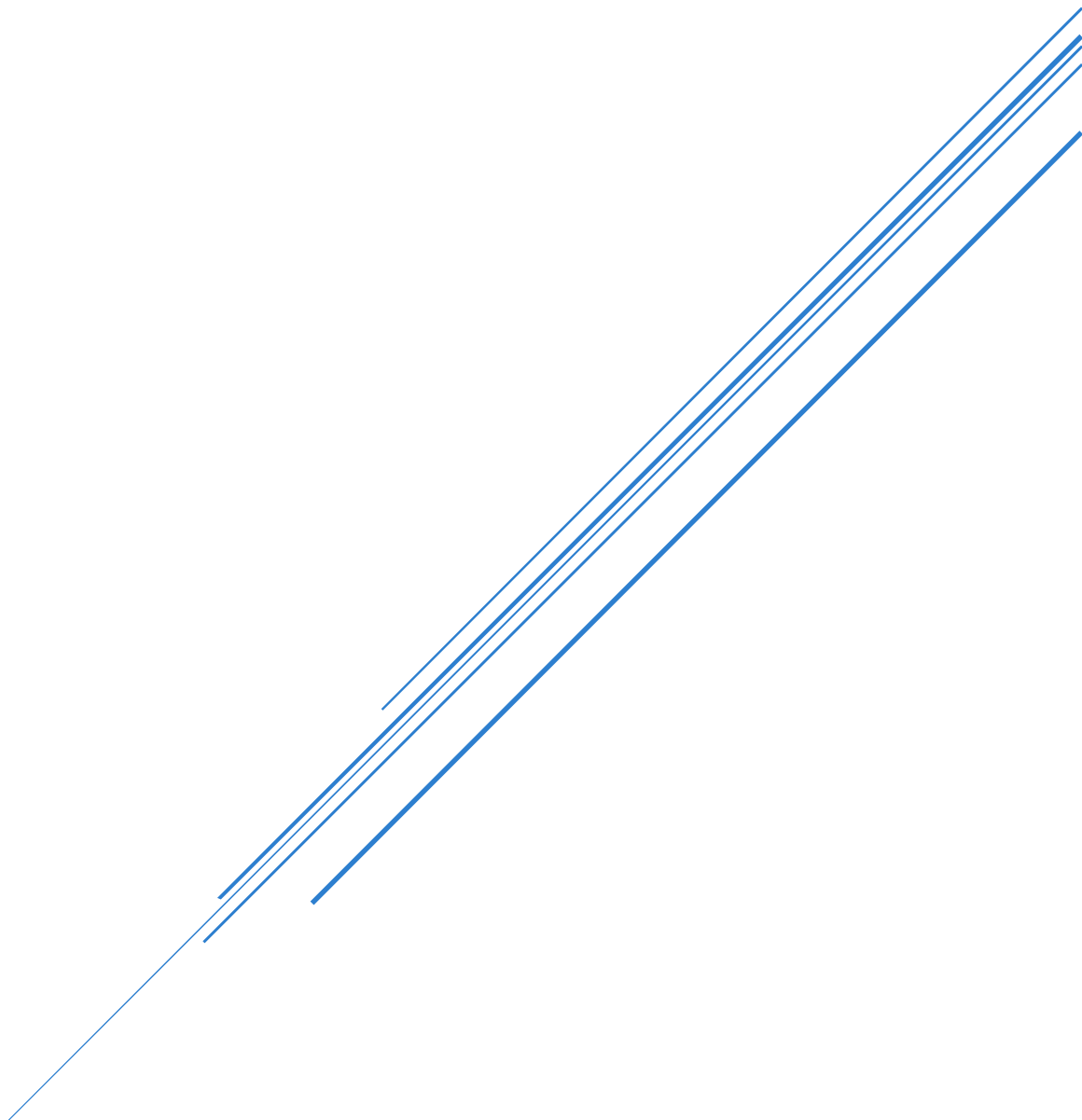
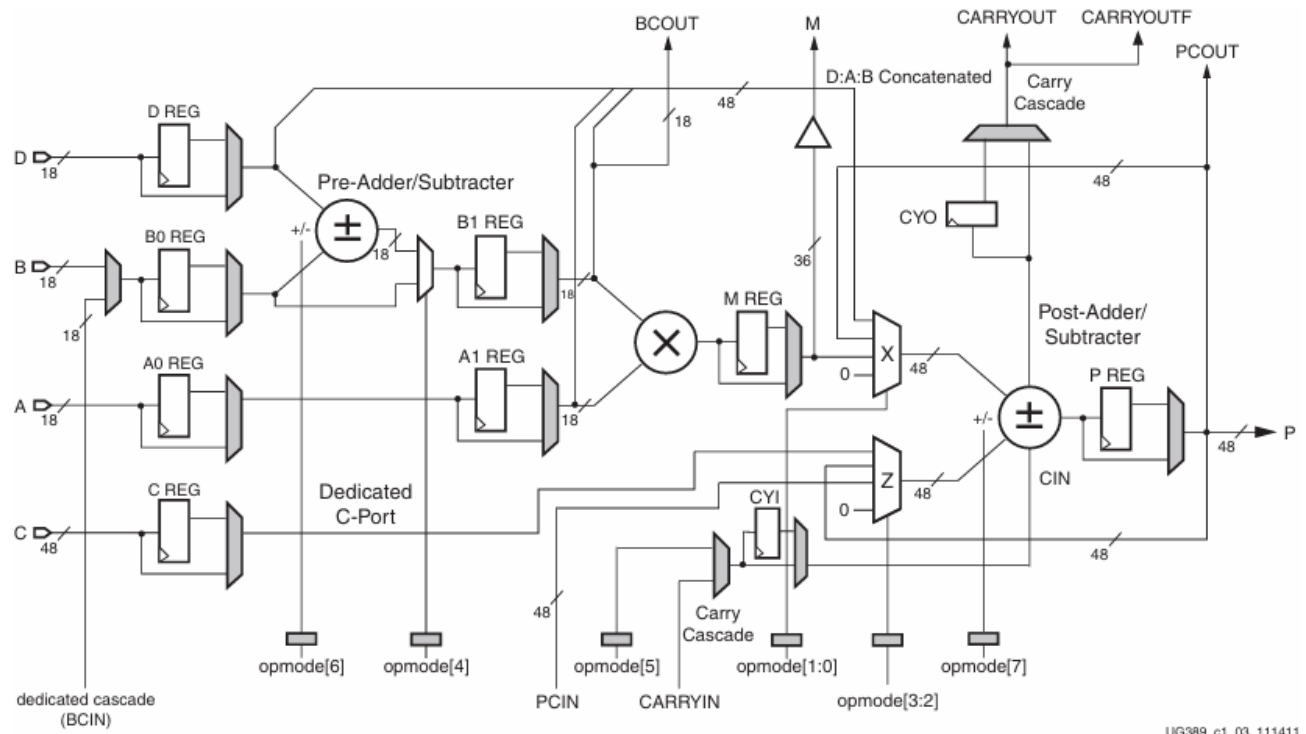


SPARTAN6 - DSP48A1

By : Mazen Mohamed Hemdan





UG389_c1_03_111411

Source of the requirements: [LINK](#)

1) RTL CODE :

- Port and Parameters Initialization

```
1  module DSP48A1 #(
2      parameter A0REG = 0,
3      parameter A1REG = 1,
4      parameter B0REG = 0,
5      parameter B1REG = 1,
6      parameter CREG = 1,
7      parameter DREG = 1,
8      parameter MREG = 1,
9      parameter PREG = 1,
10     parameter CARRYINREG = 1,
11     parameter CARRYOUTREG = 1,
12     parameter OPMODEREG = 1,
13     parameter CARRYINSEL = "OPMODE5",
14     parameter B_INPUT = "DIRECT",
15     parameter RSTTYPE = "SYNC"
16 ) (
17     input clk, CARRYIN,
18     input CEA, CEB, CEC, CECARRYIN, CED, CEM, CEOPMODE, CEP,
19     input RSTA, RSTB, RSTC, RSTCARRYIN, RSTD, RSTM, RSTOPMODE, RSTP,
20     input [17:0] A,B,D,BCIN,
21     input [47:0] C,PCIN,
22     input [7:0] opmode,
23     output CARRYOUT,CARRYOUTF,
24     output [17:0] BCOUT,
25     output [35:0] M,
26     output [47:0] P,PCOUT
27 );
```

- Wires

```
1  wire [17:0] B_mux,mux_D_out,mux_B0_out,mux_A0_out,pre_AddSub_out,B1,mux_A1_out;
2  wire [47:0] mux_C_out;
3  wire [7:0] mux_opmode_out;
4  wire carrycascade,mux_CYI_out,post_CARRYOUT;
5  wire [35:0] M_out;
6  wire [47:0] post_AddSub_out;
7  reg [47:0] X,Z;
```

- All 5 Stages

```
1 // Stage 1
2 assign B_mux = (B_INPUT == "CASCADE")? BCIN : B;
3 Reg_Mux #(RSTTYPE,DREG) D_Reg (clk,RSTD,CED,D,mux_D_out);
4 Reg_Mux #(RSTTYPE,B0REG) B0_Reg (clk,RSTB,CEB,B_mux,mux_B0_out);
5 Reg_Mux #(RSTTYPE,A0REG) A0_Reg (clk,RSTA,CEA,A,mux_A0_out);
6 Reg_Mux #(RSTTYPE,CREG,48) C_Reg (clk,RSTC,CEC,C,mux_C_out);
7 Reg_Mux #(RSTTYPE,OPMODEREG,8) opmode_reg (clk,RSTOPMODE,CEOPMODE,opmode,mux_opmode_out);
8
9 // Stage 2
10 AddSub Pre_AddSub (mux_D_out,mux_B0_out,0,mux_opmode_out[6],pre_AddSub_out);
11 assign B1 = (mux_opmode_out[4])? pre_AddSub_out : mux_B0_out;
12 Reg_Mux #(RSTTYPE,B1REG) B1_Reg (clk,RSTB,CEB,B1,BCOUT);
13 Reg_Mux #(RSTTYPE,A1REG) A1_Reg (clk,RSTA,CEA,mux_A0_out,mux_A1_out);
14
15 // Stage 3
16 MULT mult(BCOUT,mux_A1_out,M_out);
17 Reg_Mux #(RSTTYPE,MREG,36) M_Reg (clk,RSTM,CEM,M_out,M);
18 assign carrycascade = (CARRYINSEL == "OPMODE5")? mux_opmode_out[5] : CARRYIN ;
19 Reg_Mux #(RSTTYPE,CARRYINREG,1) CYI (clk,RSTCARRYIN,CECARRYIN,carrycascade,mux_CYI_out);
20
21 // Stage 4
22 always@(*)begin
23     case(mux_opmode_out[1:0])
24         2'b11 : X = {mux_D_out[11:0],mux_A1_out,BCOUT};
25         2'b10 : X = PCOUT;
26         2'b01 : X = {12'b0, M};
27         2'b00 : X = 0;
28         default : X = 0;
29     endcase
30 end
31 always@(*)begin
32     case(mux_opmode_out[3:2])
33         2'b11 : Z = mux_C_out;
34         2'b10 : Z = PCOUT;
35         2'b01 : Z = PCIN;
36         2'b00 : Z = 0;
37         default : Z = 0;
38     endcase
39 end
40
41 // Stage 5
42 AddSub #(48) Post_AddSub (Z,X,mux_CYI_out,mux_opmode_out[7],post_AddSub_out,post_CARRYOUT);
43 Reg_Mux #(RSTTYPE,CARRYOUTREG,1) CYO (clk,RSTCARRYIN,CECARRYIN,post_CARRYOUT,CARRYOUT);
44 assign CARRYOUTF = CARRYOUT;
45 Reg_Mux #(RSTTYPE,PREG,48) P_Reg (clk,RSTP,CEP,post_AddSub_out,P);
46 assign PCOUT = P;
47
48 endmodule
```

- Register followed by Mux

```

1  module Reg_Mux(clk,rst,CE,d,out);
2  parameter RSTTYPE = "SYNC";
3  parameter REG = 1;
4  parameter WIDTH = 18;
5  input clk,rst,CE;
6  input [WIDTH-1:0]d;
7  reg [WIDTH-1:0]out_reg;
8  output [WIDTH-1:0]out;
9  generate
10     if(RSTTYPE == "SYNC")begin
11         always@(posedge clk)begin
12             if(rst)out_reg<=0;
13             else if(CE)out_reg<=d;
14         end
15     end
16     else if(RSTTYPE == "ASYN")begin
17         always@(posedge clk or posedg rst)begin
18             if(rst)out_reg<=0;
19             else if(CE)out_reg<=d;
20         end
21     end
22 endgenerate
23 assign out = (REG)? out_reg : d;
24 endmodule

```

- Add/Sub

```

1  module AddSub(in0,in1,cin,opmode,sum,carryout);
2  parameter WIDTH = 18;
3  input [WIDTH-1:0]in0,in1;
4  input opmode,cin;
5  output [WIDTH-1:0]sum;
6  output carryout;
7  assign {carryout,sum} = (opmode)? (in0-(in1+cin)) : (in0+in1+cin);
8  endmodule

```

- Multiplier

```

1  module MULT(in0,in1,out);
2  parameter WIDTH = 18;
3  input [WIDTH-1:0]in0,in1;
4  output [(WIDTH*2)-1:0]out;
5  assign out = in0 * in1;
6  endmodule

```

2) TESTBENCH CODE :

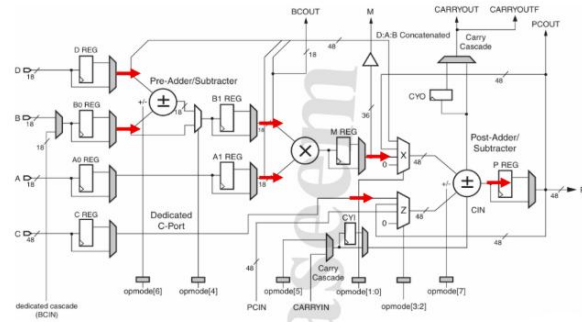
- module instantiation and all required reg and wires

```
1  module DSP48A1_tb;
2      reg clk;
3      reg CARRYIN;
4      reg CEA, CEB, CEC, CECARRYIN, CED, CEM, CEOPMODE, CEP;
5      reg RSTA, RSTB, RSTC, RSTCARRYIN, RSTD, RSTM, RSTOPMODE, RSTP;
6      reg [17:0] A, B, D, BCIN;
7      reg [47:0] C, PCIN;
8      reg [7:0] opmode;
9      wire CARRYOUT, CARRYOUTF;
10     wire [17:0] BCOUT;
11     wire [35:0] M;
12     wire [47:0] P, PCOUT;
13     reg CARRYOUT_old;
14     reg [47:0] P_old;
15     DSP48A1 DUT (
16         .clk(clk),
17         .CARRYIN(CARRYIN),
18         .CEA(CEA), .CEB(CEB), .CEC(CEC), .CECARRYIN(CECARRYIN),
19         .CED(CED), .CEM(CEM), .CEOPMODE(CEOPMODE), .CEP(CEP),
20
21         .RSTA(RSTA), .RSTB(RSTB), .RSTC(RSTC), .RSTCARRYIN(RSTCARRYIN),
22         .RSTD(RSTD), .RSTM(RSTM), .RSTOPMODE(RSTOPMODE), .RSTP(RSTP),
23
24         .A(A), .B(B), .D(D), .BCIN(BCIN),
25         .C(C), .PCIN(PCIN),
26         .opmode(opmode),
27
28         .CARRYOUT(CARRYOUT), .CARRYOUTF(CARRYOUTF),
29         .BCOUT(BCOUT),
30         .M(M),
31         .P(P), .PCOUT(PCOUT)
32     );
```

- CLK configuration and verify reset operation

```
1 initial begin
2     clk = 0;
3     forever #1 clk = ~clk;
4 end
5 // 2. Stimulus Generation
6 initial begin
7     // 2.1. Verify Reset Operation
8     // Assert all resets
9     RSTA = 1; RSTB = 1; RSTC = 1; RSTCARRYIN = 1;
10    RSTD = 1; RSTM = 1; RSTOPMODE = 1; RSTP = 1;
11
12    // Drive remaining inputs with random values
13    A = $random; B = $random; D = $random; BCIN = $random;
14    C = $random; PCIN = $random; opmode = $random; CARRYIN = $random;
15    CEA = $random; CEB = $random; CEC = $random; CECARRYIN = $random;
16    CED = $random; CEM = $random; CEOPMODE = $random; CEP = $random;
17
18    // Wait for negative edge of the clock
19    @(negedge clk);
20
21    // Self Checking
22    if (CARRYOUT != 0 || CARRYOUTF != 0 || BCOUT != 0 ||
23        M != 0 || P != 0 || PCOUT != 0) begin
24        $display("Outputs are not zero during reset");
25        $stop;
26    end
27
28    // Deassert all resets
29    RSTA = 0; RSTB = 0; RSTC = 0; RSTCARRYIN = 0;
30    RSTD = 0; RSTM = 0; RSTOPMODE = 0; RSTP = 0;
31    CEA = 1; CEB = 1; CEC = 1; CECARRYIN = 1;
32    CED = 1; CEM = 1; CEOPMODE = 1; CEP = 1;
```

- Verify DSP Path 1

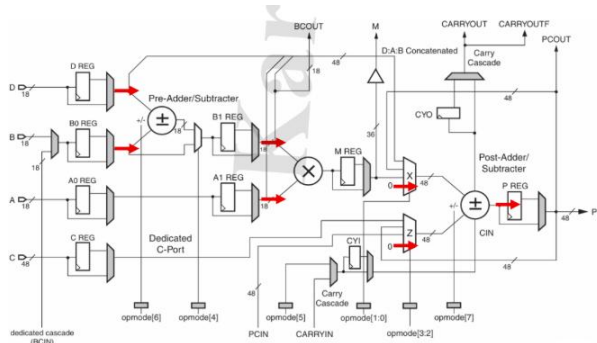


```

1 // 2.2. Verify DSP Path 1
2 opmode = 8'b11011101;
3 A = 20; B = 10; C = 350; D = 25;
4 BCIN = $random; PCIN = $random; CARRYIN = $random;
5 repeat(4)@(negedge clk);
6 if (BCOUT !== 18'hf || M !== 36'h12c || P !== 48'h32 ||
7     PCOUT !== 48'h32 || CARRYOUT !== 1'b0 || CARRYOUTF !== 1'b0) begin
8     $display("Outputs are wrong for path 1");
9     $stop;
10    end

```

- Verify DSP Path 2

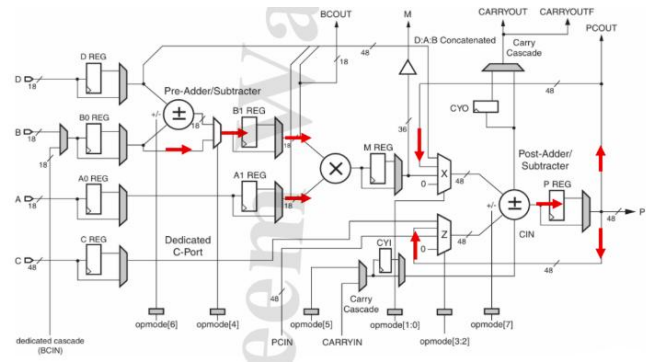


```

1 // 2.3. Verify DSP Path 2
2 opmode = 8'b00010000;
3 A = 20; B = 10; C = 350; D = 25;
4 BCIN = $random; PCIN = $random; CARRYIN = $random;
5 repeat(3)@(negedge clk);
6 if (BCOUT !== 18'h23 || M !== 36'h2bc || P !== 48'h0 ||
7     PCOUT !== 48'h0 || CARRYOUT !== 1'b0 || CARRYOUTF !== 1'b0) begin
8     $display("Outputs are wrong for path 2");
9     $stop;
10    end

```


- Verify DSP Path 3

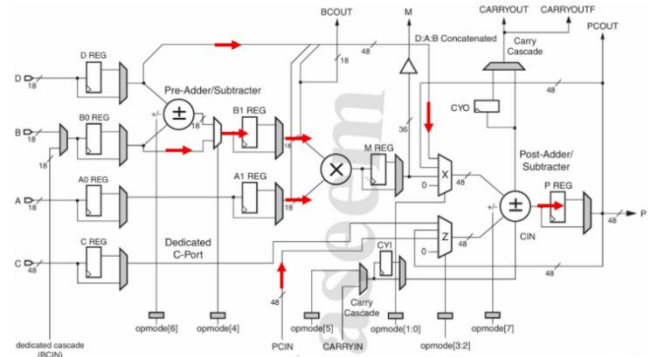


```

1 // 2.4. Verify DSP Path 3
2 opmode = 8'b00001010;
3 A = 20; B = 10; C = 350; D = 25;
4 BCIN = $random; PCIN = $random; CARRYIN = $random;
5 P_old = P; CARRYOUT_old = CARRYOUT;
6 repeat(3)@(negedge clk);
7 if (BCOUT !== 18'ha || M !== 36'hc8 || P !== P_old ||
8     PCOUT !== P_old || CARRYOUT !== CARRYOUT_old || CARRYOUTF !== CARRYOUT_old) begin
9     $display("Outputs are wrong for path 3");
10    $stop;
11    end

```

- Verify DSP Path 4



```

1 // 2.5. Verify DSP Path 4
2 opmode = 8'b10100111;
3 A = 5; B = 6; C = 350; D = 25; PCIN = 3000;
4 BCIN = $random; CARRYIN = $random;
5 P_old = P; CARRYOUT_old = CARRYOUT;
6 repeat(3)@(negedge clk);
7 if (BCOUT !== 18'h6 || M !== 36'h1e || P !== 48'hfe6ffec0bb1 ||
8     PCOUT !== 48'hfe6ffec0bb1 || CARRYOUT !== 1 || CARRYOUTF !== 1) begin
9     $display("Outputs are wrong for path 4");
10    $stop;
11    end
12
13    $stop;
14    end
15
16 endmodule

```

3) DO FILE :

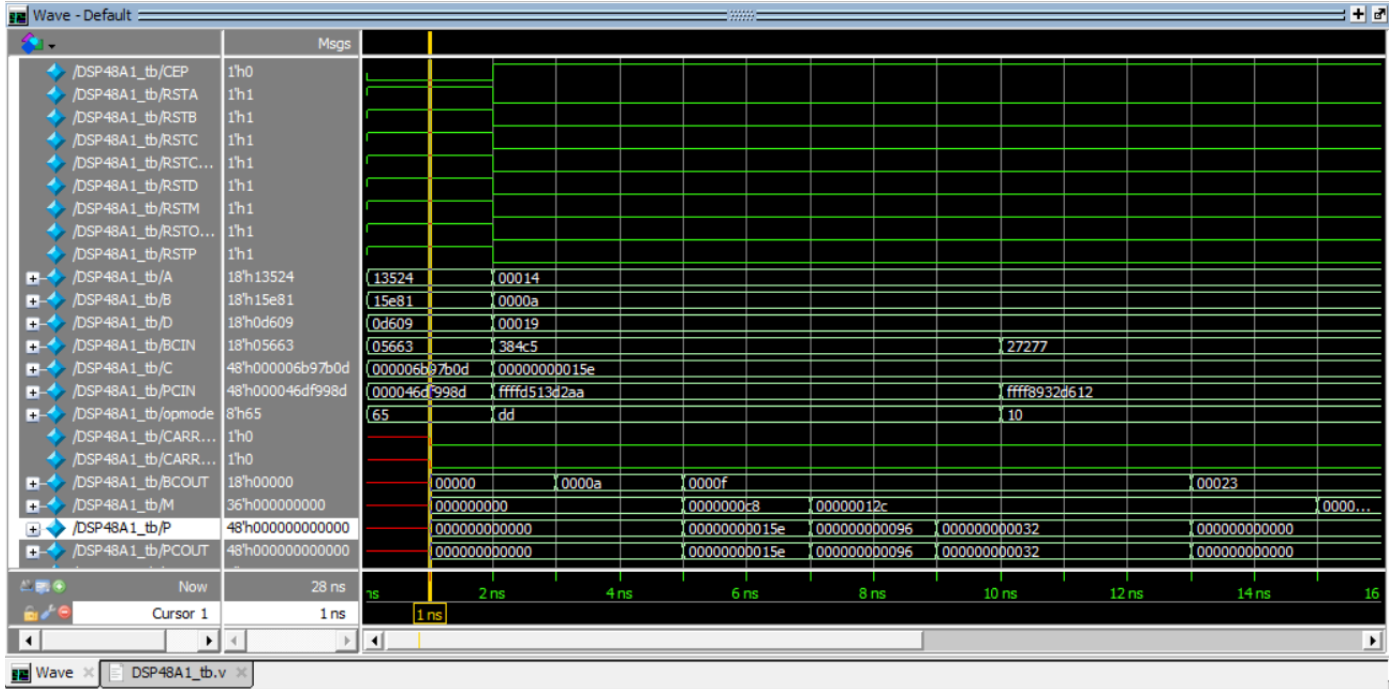


```
1  vlib work
2  vlog DSP48A1.v DSP48A1_tb.v Reg_Mux.v AddSub.v MULT.v
3  vsim -voptargs=+acc work.DSP48A1_tb
4  add wave *
5  run -all
6  #quit -sim
```

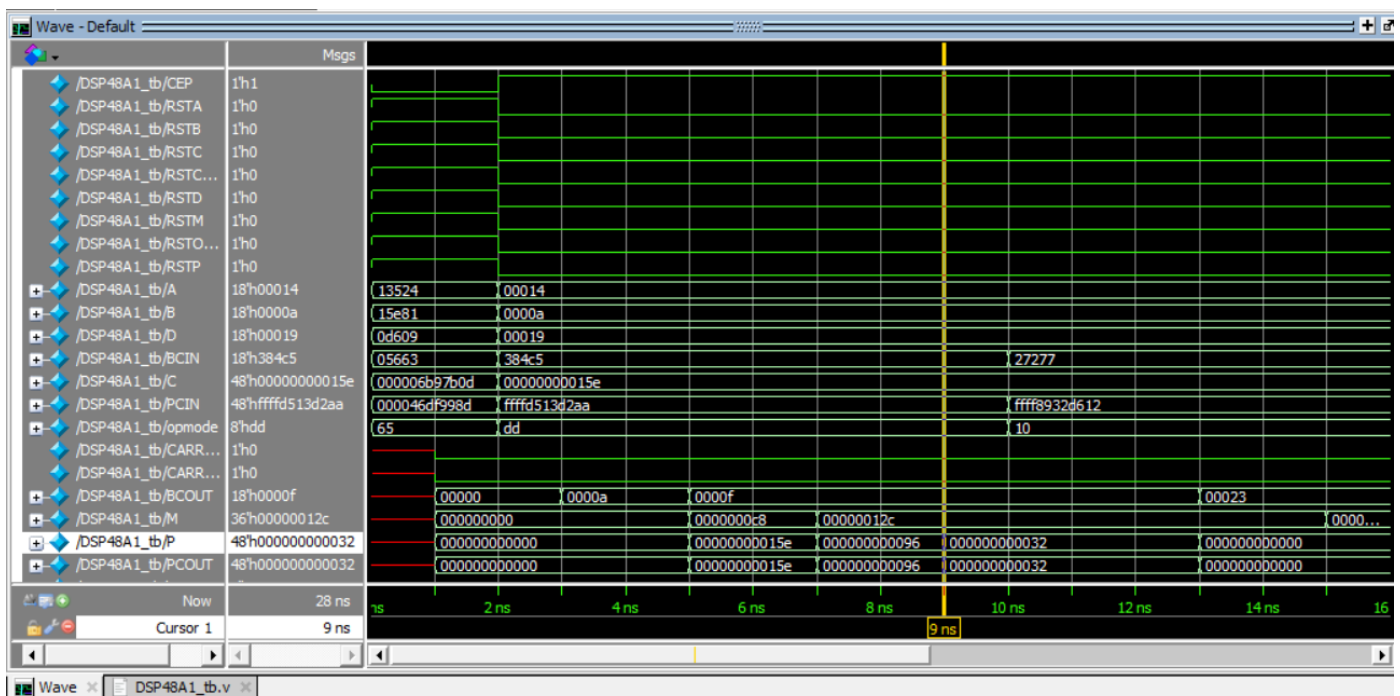
```
VSIM 13> do RUN_DSP48A1.do
# ** Warning: (vlib-34) Library already exists at "work".
# Errors: 0, Warnings: 1
# QuestaSim-64 vlog 2021.1 Compiler 2021.01 Jan 19 2021
# Start time: 04:32:25 on Jul 25,2025
# vlog -reportprogress 300 DSP48A1.v DSP48A1_tb.v Reg_Mux.v AddSub.v MULT.v
# -- Compiling module DSP48A1
# -- Compiling module DSP48A1_tb
# -- Compiling module Reg_Mux
# -- Compiling module AddSub
# -- Compiling module MULT
#
# Top level modules:
#     DSP48A1_tb
# End time: 04:32:25 on Jul 25,2025, Elapsed time: 0:00:00
# Errors: 0, Warnings: 0
```

4) QUESTASIM SNIPPETS :

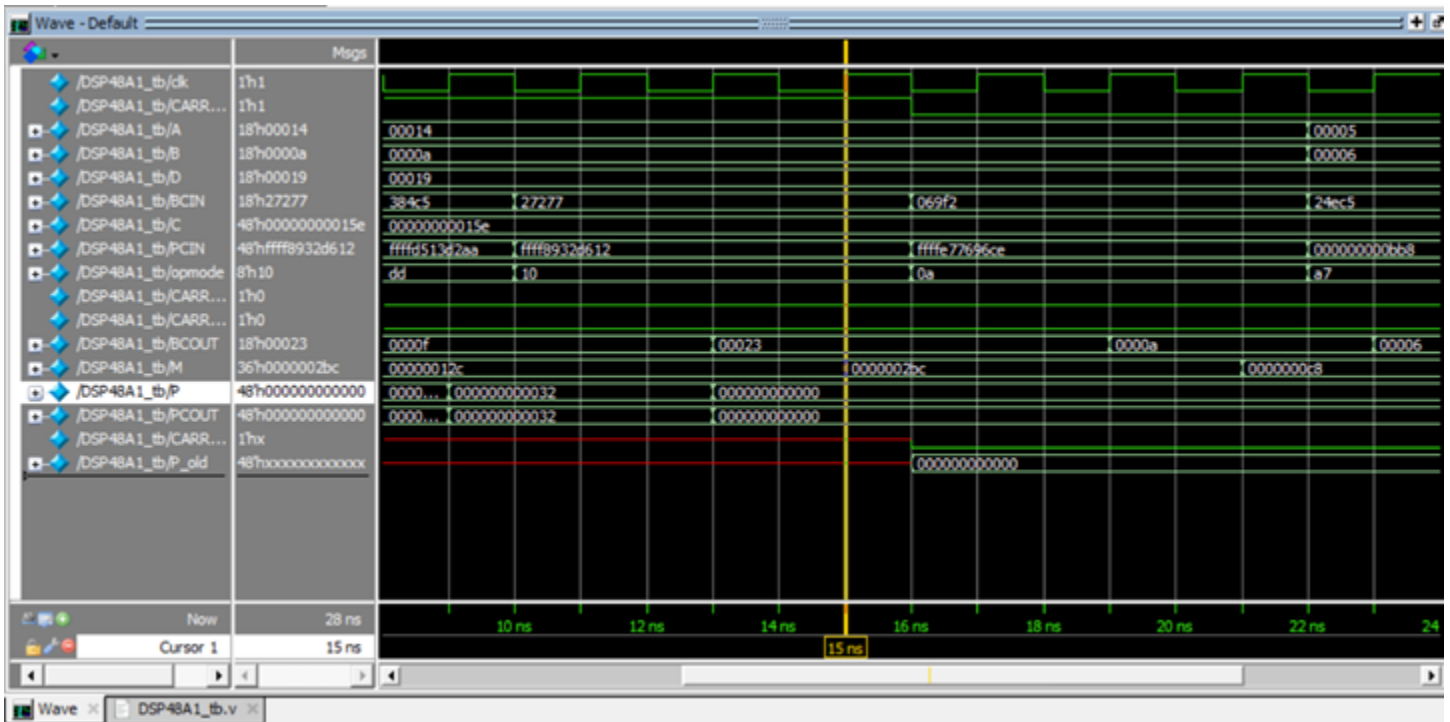
- 2.1. Verify Reset Operation



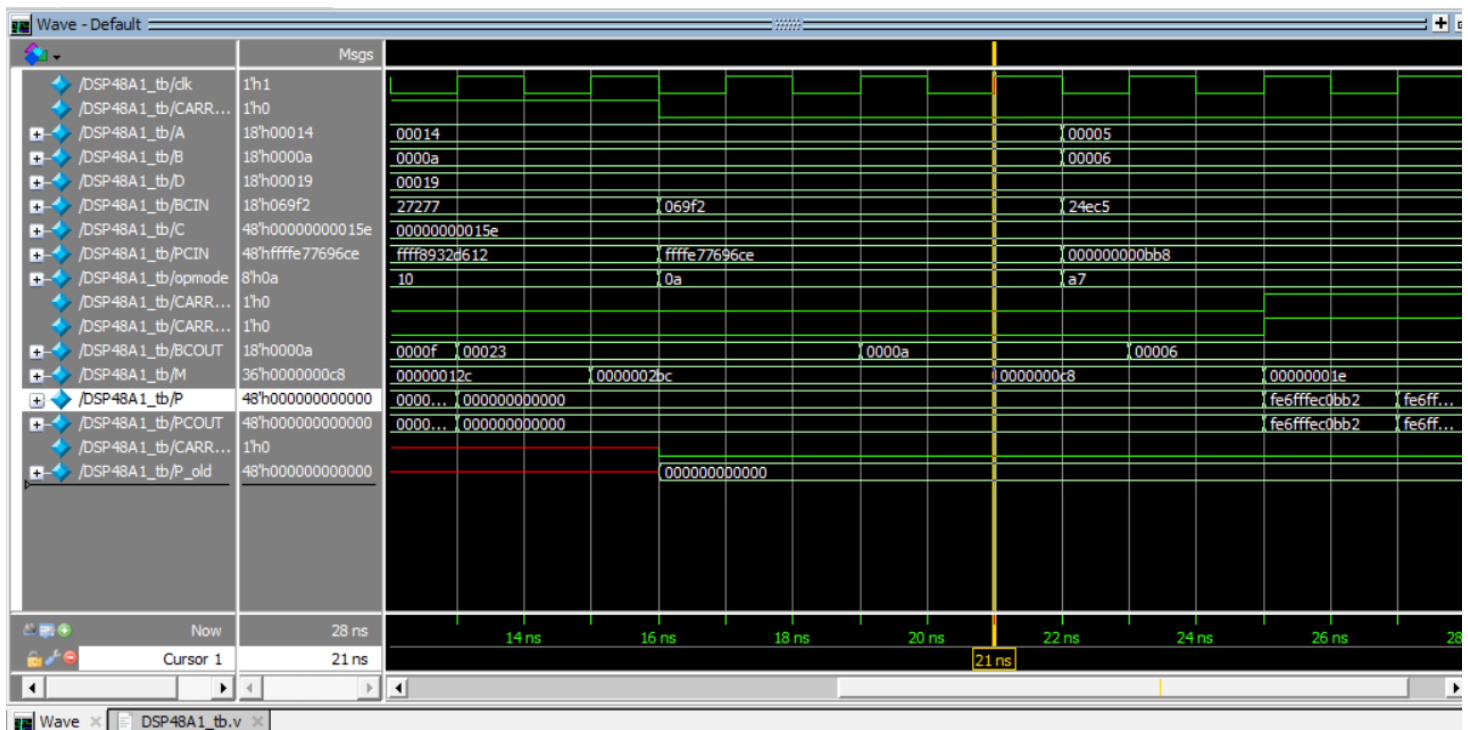
- 2.2. Verify DSP Path 1



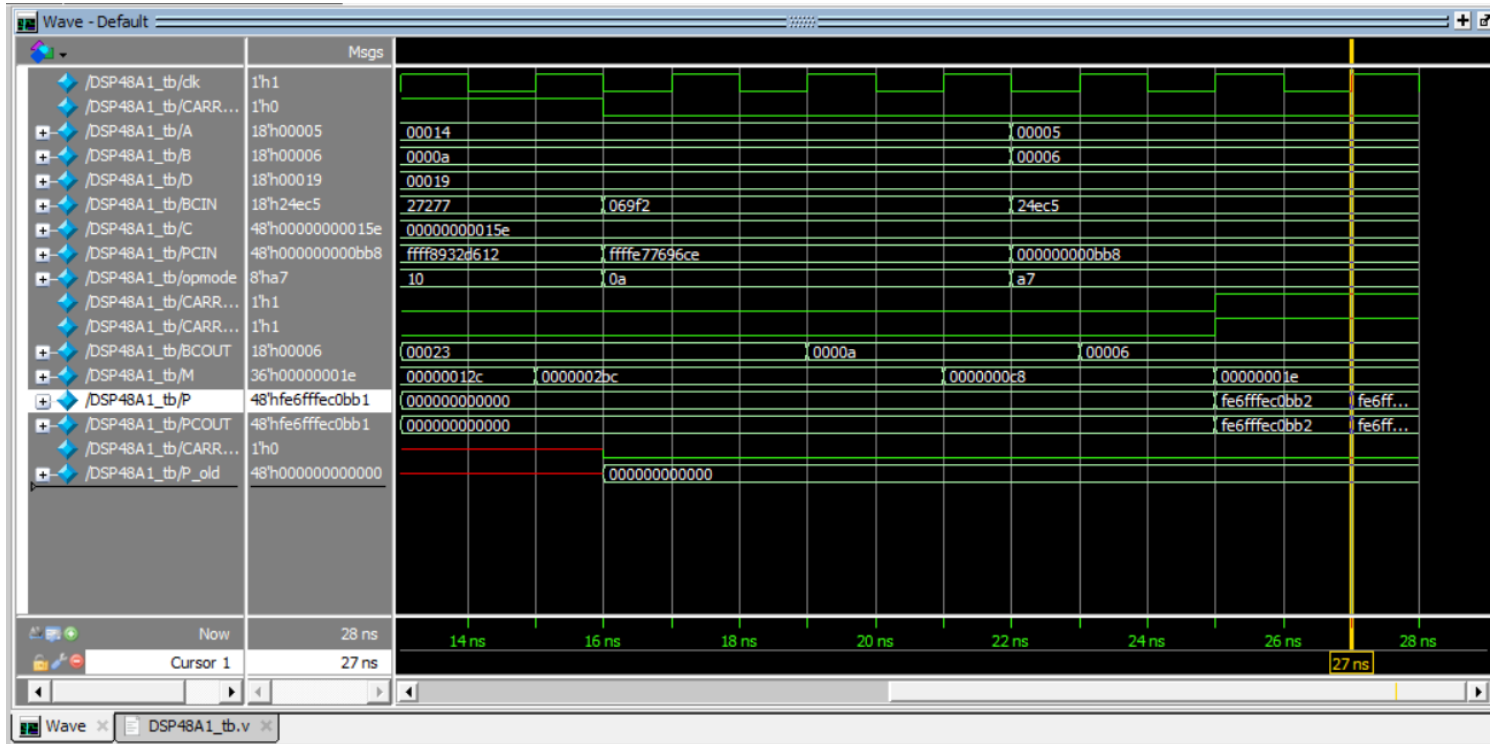
- 2.3. Verify DSP Path 2
- Removed the rst and CE ports for more clear view



- 2.4. Verify DSP Path 3



- 2.5. Verify DSP Path 4



5) CONSTRAINT FILE :



```
1  ## Clock signal
2  set_property -dict { PACKAGE_PIN W5    IOSTANDARD LVCMOS33 } [get_ports clk]
3  create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports clk]
```

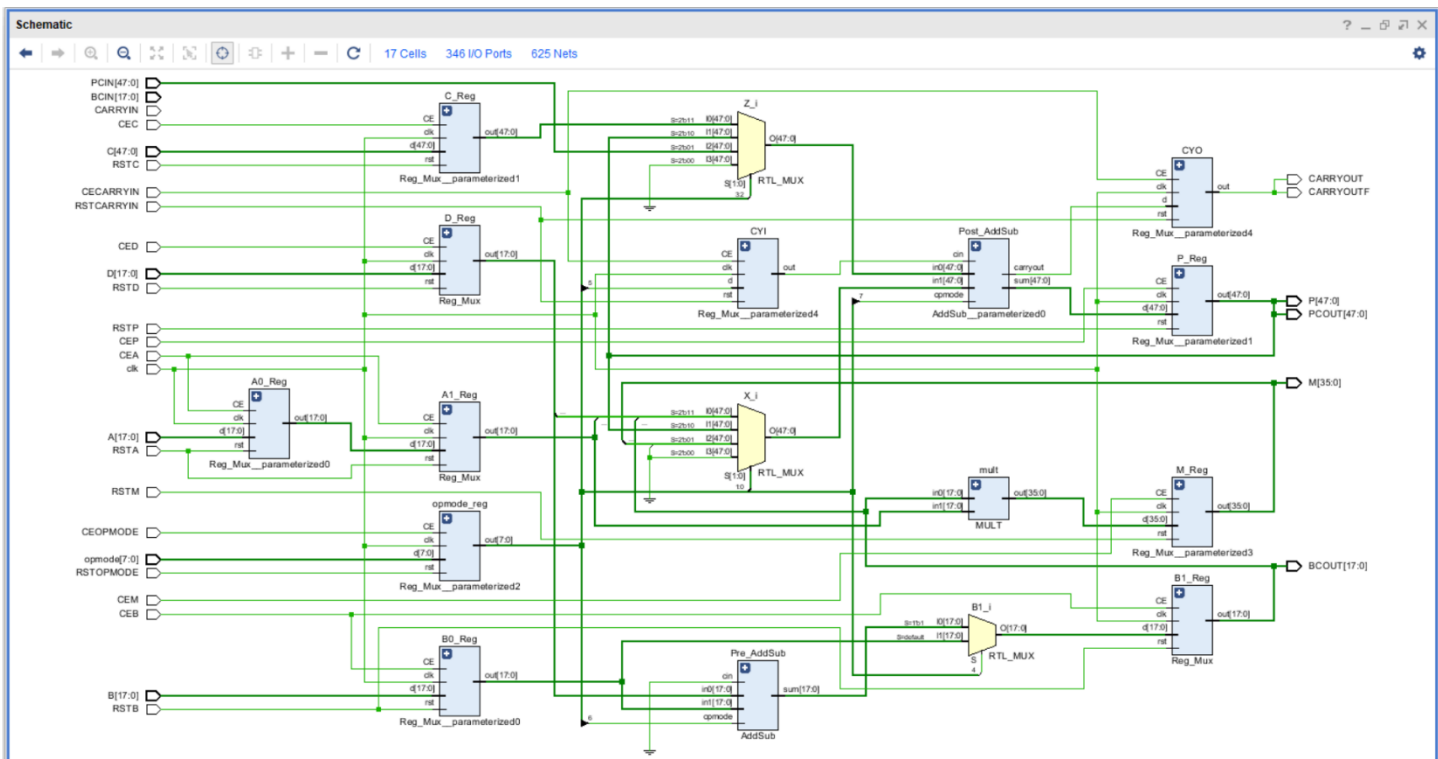
- Defines the clock frequency to 100 MHz to pin W5

6) ELABORATION:

- Messages



- Schematic Snippets



7) SYNTHESIS :

- Messages

The Messages window displays the following content:

- Vivado Commands (3 infos)**
 - General Messages (3 infos)
 - [IP_Flow 19-234] Refreshing IP repositories
 - [IP_Flow 19-1704] No user IP repositories specified
 - [IP_Flow 19-2313] Loaded Vivado IP repository 'C:/Xilinx/Vivado/2018.2/data/ip'.
- Synthesis (44 warnings, 40 infos)**
 - [Common 17-349] Got license for feature 'Synthesis' and/or device 'xc7a200t'
 - [Synth 8-2490] overwriting previous definition of module DSP48A1 [DSP48A1.v.1]
 - [Synth 8-6157] synthesizing module 'DSP48A1' [DSP48A1.v.1] (9 more like this)

- Utilization Report

The Utilization Report window shows the following hierarchy and resource usage:

Name	Slice LUTs (134600)	Slice Registers (269200)	DSPs (740)	Bonded IOB (500)	BUFGCTRL (32)
DSP48A1	230	160	1	327	1
A1_Reg (Reg_Mux)	0	18	0	0	0
B1_Reg (Reg_Mux_0)	0	18	0	0	0
C_Reg (Reg_Mux_pa...)	0	48	0	0	0
CYI (Reg_Mux_para...)	1	1	0	0	0
CYO (Reg_Mux_para...)	0	1	0	0	0
D_Reg (Reg_Mux_2)	36	18	0	0	0
M_Reg (Reg_Mux_pa...)	0	0	1	0	0
opmode_reg (Reg_Mu...)	193	8	0	0	0
P_Reg (Reg_Mux_pa...)	0	48	0	0	0
Post_AddSub (AddSub...)	0	0	0	0	0
Pre_AddSub (AddSub)	0	0	0	0	0

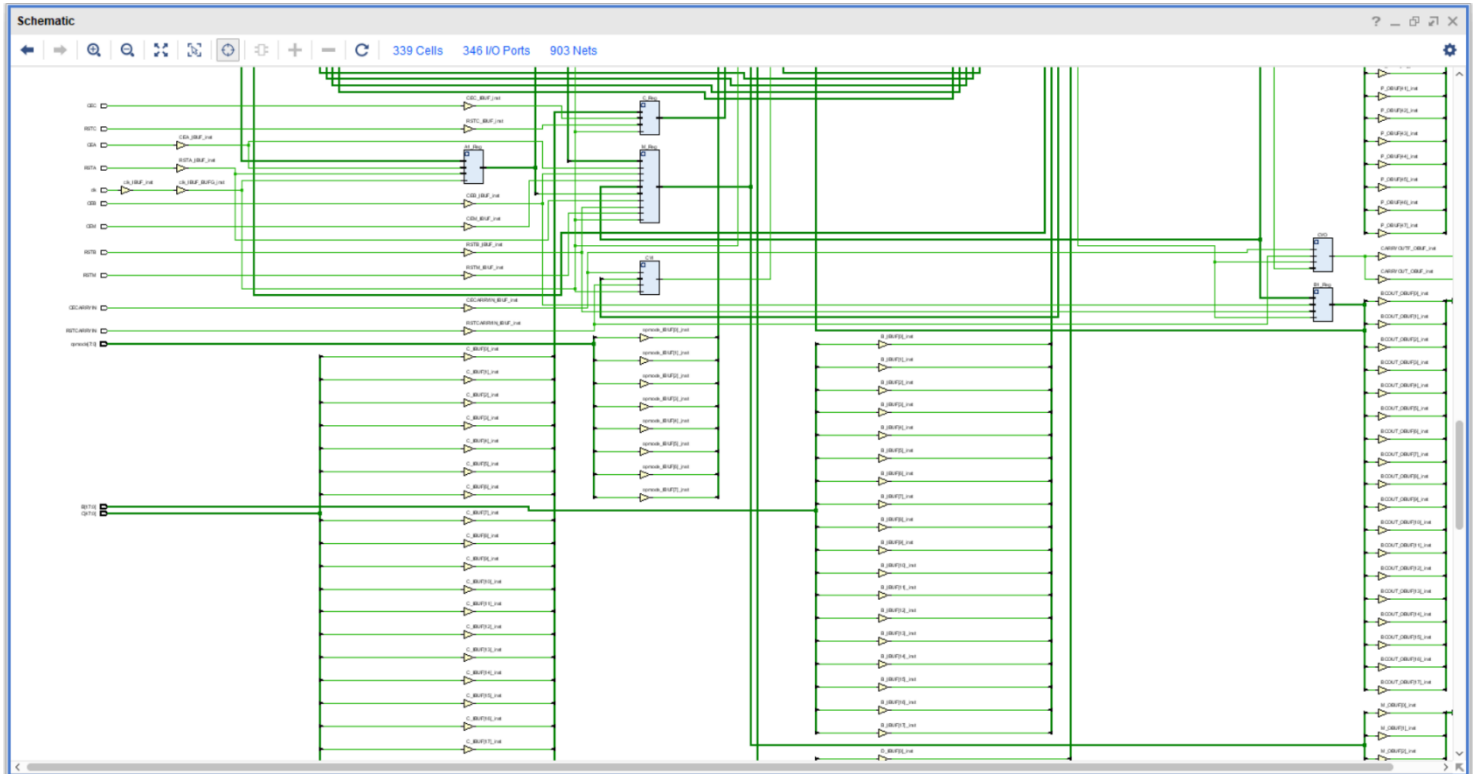
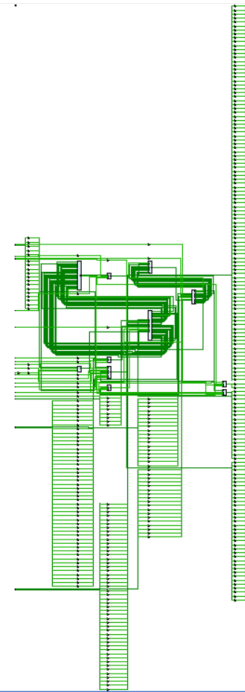
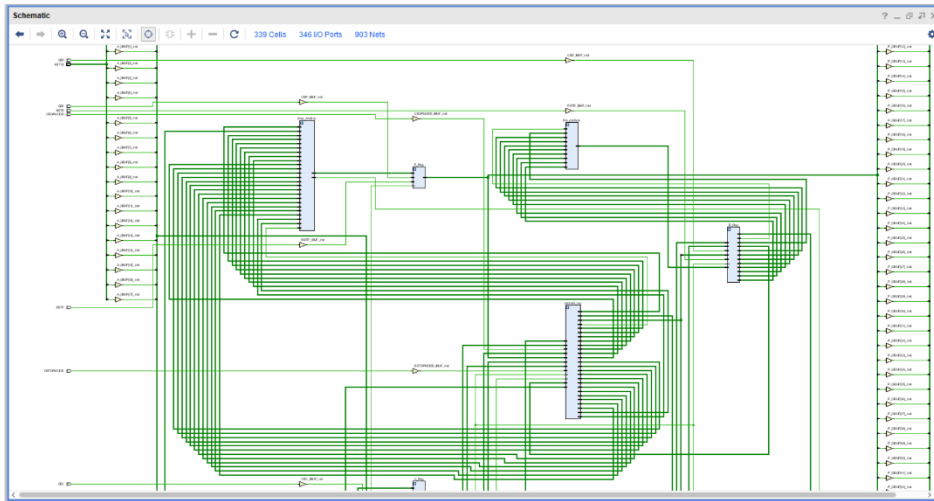
- Timing Report

The Design Timing Summary window displays the following information:

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 5.168 ns	Worst Hold Slack (WHS): 0.182 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 106	Total Number of Endpoints: 106	Total Number of Endpoints: 162

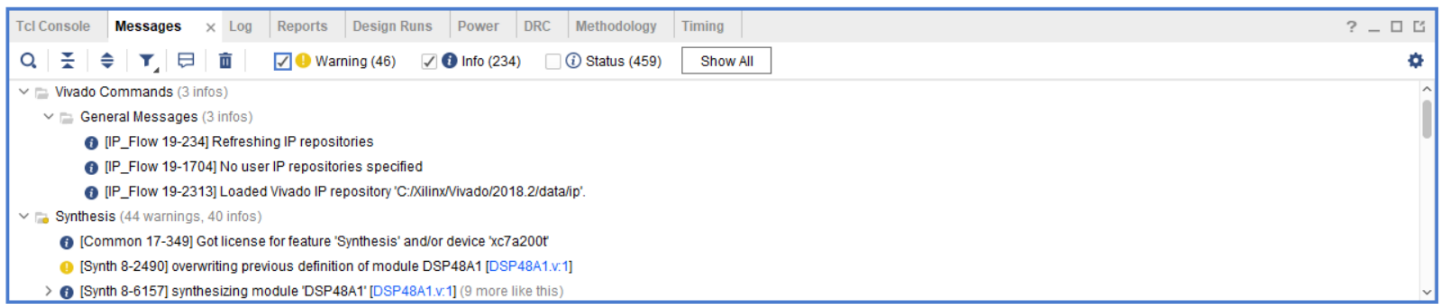
All user specified timing constraints are met.

- Schematic Snippets



8) IMPLEMENTATION :

- Messages



- Utilization Report

The Utilization Report window displays the following hierarchy and utilization data:

Name	Slice LUTs (133800)	Slice Registers (267600)	Slice (33450)	LUT as Logic (133800)	LUT Flip Flop Pairs (133800)	DSPs (740)	Bonded IOB (500)	BUFGCTRL (32)
DSP48A1	230	179	106	230	54	1	327	1
A1_Reg (Reg_Mux)	0	18	6	0	0	0	0	0
B1_Reg (Reg_Mux_0)	0	36	14	0	0	0	0	0
C_Reg (Reg_Mux_pa...)	0	48	16	0	0	0	0	0
CYI (Reg_Mux_para...)	1	1	1	1	1	0	0	0
CYO (Reg_Mux_para...)	0	2	2	0	0	0	0	0
D_Reg (Reg_Mux_2)	36	18	18	36	0	0	0	0
M_Reg (Reg_Mux_pa...)	0	0	0	0	0	1	0	0
opmode_reg (Reg_Mu...)	193	8	61	193	0	0	0	0
P_Reg (Reg_Mux_pa...)	0	48	12	0	0	0	0	0
Post_AddSub (AddSub...)	0	0	13	0	0	0	0	0
Pre_AddSub (AddSub)	0	0	5	0	0	0	0	0

The left sidebar shows the hierarchy of resources:

- Hierarchy**
- Summary**
- Slice Logic**
 - Slice LUTs (<1%)
 - LUT as Logic (<1%)
 - Slice Registers (<1%)
 - Register as Flip Flop (<1%)
 - Slice Logic Distribution
 - Slice (1%)
 - SLICEM
 - SLICEL
 - LUT Flip Flop Pairs (<1%)
 - LUT-FF pairs with one
 - LUT-FF pairs with one
 - LUT as Logic (<1%)
 - using O5 and O6
 - using O6 output only
- Memory**
- DSP**
 - DSPs (<1%)
 - DSP48E1 only
- IO and GT Specific**
 - Bonded IOB (65%)
 - IOB Slave Pads
 - IOB Master Pads
- Clocking**

- Timing Report

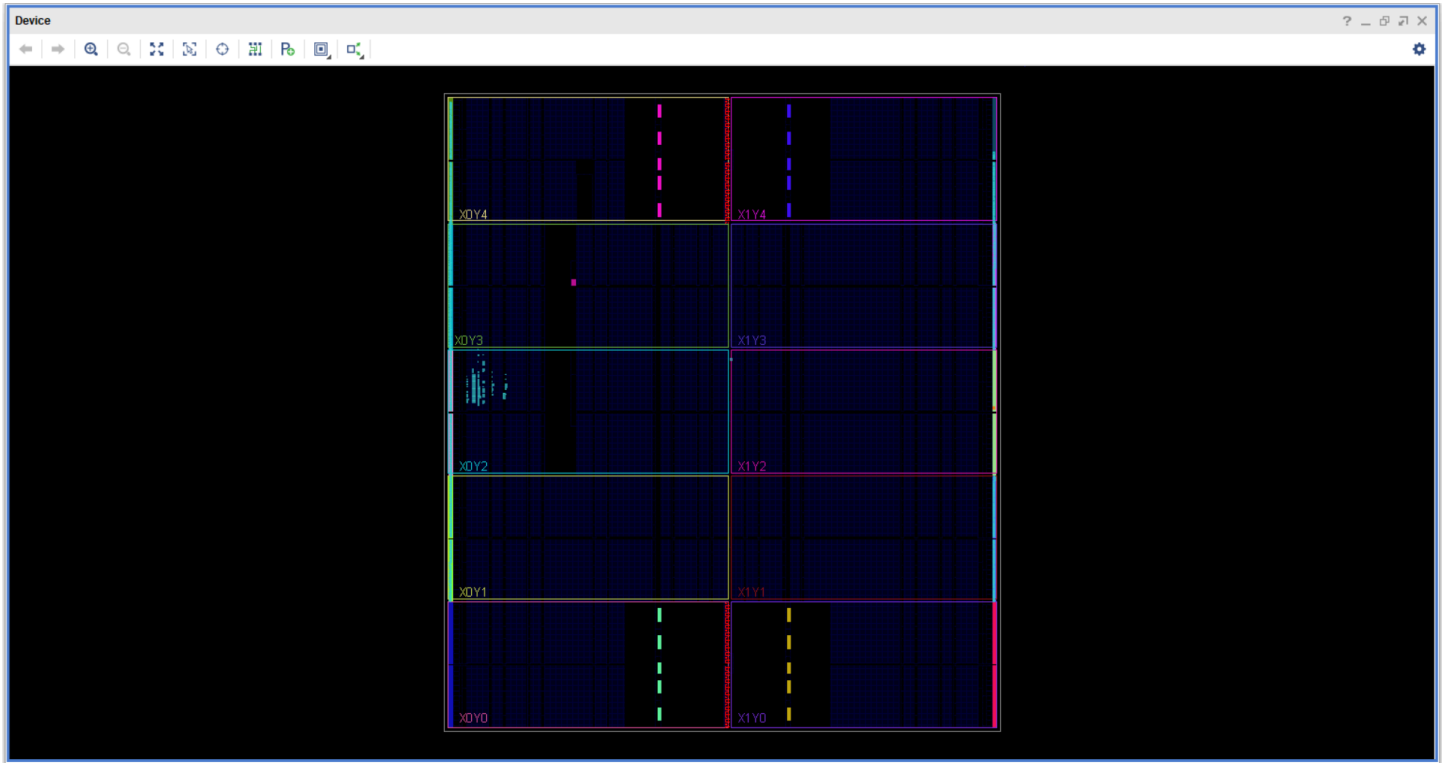
The Timing Report window displays the following Design Timing Summary:

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 3.399 ns	Worst Hold Slack (WHS): 0.266 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 125	Total Number of Endpoints: 125	Total Number of Endpoints: 181

The left sidebar shows the hierarchy of reports:

- General Information**
- Timer Settings**
- Design Timing Summary**
- Clock Summary (1)**
- Check Timing (326)**
- Intra-Clock Paths**

- Device Snippets



9) LINTING :

- 0 Warnings, 0 Errors only 9 Info which is Waived

The screenshot shows the Xilinx IDE with the DSP48A1 module open. The main editor displays the Verilog code for the DSP48A1 module, which is divided into three stages. The lint summary on the right shows 9 Info messages, all of which are waived. The lint checks panel at the bottom shows 0 Total lint errors and 0 Selected lint errors.

```
// Stage 1
assign B_mux = (B_INPUT == "CASCADE")? BCIN : B;
DIRECT
Reg_Mux #(RSTTYPE,DREG) D_Reg (clk, RSTD, CED, D, mux_D_out);
SYNC 1
Reg_Mux #(RSTTYPE,B0REG) B0_Reg (clk, RSTB, CEB, B_mux, mux_B0_out);
SYNC 0
Reg_Mux #(RSTTYPE,A0REG) A0_Reg (clk, RSTA, CEA, A, mux_A0_out);
SYNC 0
Reg_Mux #(RSTTYPE,CREG,48) C_Reg (clk, RSTC, CEC, C, mux_C_out);
SYNC 1
Reg_Mux #(RSTTYPE,OPMODEREG,8) opmode_reg (clk, RSTOPMODE, CEOPMODE, opmode, mux_opmode_out);
SYNC 1

// Stage 2
AddSub Pre_AddSub (mux_D_out, mux_B0_out, 0, mux_opmode_out[6], pre_AddSub_out);
assign B1 = (mux_opmode_out[4])? pre_AddSub_out : mux_B0_out;
Reg_Mux #(RSTTYPE,B1REG) B1_Reg (clk, RSTB, CEB, B1, BCOUT);
SYNC 1
Reg_Mux #(RSTTYPE,A1REG) A1_Reg (clk, RSTA, CEA, mux_A0_out, mux_A1_out);
SYNC 1

// Stage 3
MULT mult(BCOUT, mux_A1_out, M_out);
Reg_Mux #(RSTTYPE,MREG,36) M_Reg (clk, RSTM, CEM, M_out, M);
```

Lint Summary

Name	Count
Resolved(verified, fixed, ...)	9
Info	9

Lint Checks

Filter: Type here

Waived Fixed Pending Uninspected Bug Verified Total: 0 Selected: 0

Severity	Status	Check	Alias	Message	Module	Category	State	Owner	STARC Reference
----------	--------	-------	-------	---------	--------	----------	-------	-------	-----------------

Transcript Message Viewer Lint Checks Design Metrics Design Information Status History Lint Dashboard